

嵌入式软件开发（网络）岗位 C 语言面试题整理

一、C 语言与嵌入式基础

1. 指针与数组的区别

- 数组名是常量指针，不能赋值；指针可变
- `sizeof(array)` 是数组总大小，`sizeof(pointer)` 是指针大小
- 数组内存连续，指针可指向任意地址

2. volatile 的作用

- 防止编译器优化，保证每次访问都从内存读取
- 用于硬件寄存器、中断标志等

3. 如何防止野指针

- 初始化为 NULL
- 释放后置 NULL
- 不返回局部变量地址
- 使用内存检测工具

4. static 关键字的作用

- 局部变量：生命周期延长到程序结束
- 全局变量/函数：仅当前文件可见

5. 结构体对齐

- 按最大成员对齐
- 可用 `__attribute__((packed))` 控制
- 影响内存占用和访问效率

6. C 语言常见内存错误

- 未初始化指针
- 重复释放
- 数组越界
- 内存泄漏

7. 深拷贝与浅拷贝的区别

- 浅拷贝：只复制指针
- 深拷贝：复制实际数据

8. 常见链表操作考点

- 单链表翻转
- 检测环（快慢指针）
- 合并有序链表

- 找中间节点、倒数第 K 个节点

二、常考 C 语言基础与进阶问题（含简要答案）

1. *const* 的各种组合: *const int*、*int const* 等的区别? **

- `const int* p`: 指向常量的指针, 不能通过 `p` 修改值
- `int* const p`: 常量指针, `p` 不能指向其他地址
- `const int* const p`: 指向常量的常量指针

2. `malloc` / `calloc` / `realloc` / `free` 的区别, 内存泄漏场景?

- `malloc`: 分配未初始化内存
- `calloc`: 分配并初始化为 0
- `realloc`: 调整已分配内存大小
- `free`: 释放内存
- 泄漏场景: 未释放、重复分配未释放、丢失指针

3. C 语言中的宏和内联函数的区别?

- 宏: 预处理展开, 无类型检查
- 内联函数: 有类型检查, 可调试, 编译器决定是否内联

4. C 语言中的函数指针及其应用?

- 指向函数的指针
- 用于回调、接口、状态机等

5. C 语言中的联合体 (`union`) 和结构体 (`struct`) 区别?

- `union` 所有成员共用一块内存
- `struct` 各成员独立分配内存

6. C 语言中的位运算常见应用?

- 位标志、权限控制、数据压缩、硬件寄存器操作

7. 如何实现链表、栈、队列等基本数据结构?

- 链表: 节点结构体+指针
- 栈/队列: 数组或链表实现, 维护头尾指针

8. C 语言中如何实现多文件编译和头文件保护?

- 多文件: `.c/.h` 分离, 头文件声明, 源文件实现
- 头文件保护: `#ifndef/#define/#endif`

9. C 语言中的死锁及其避免方法?

- 死锁: 多个线程互相等待资源
- 避免: 加锁顺序一致、加超时、减少锁粒度

10. 请写出一个反转字符串的函数。

```
void reverse(char *s) {
    int i = 0, j = strlen(s) - 1;
    while(i < j) {
        char t = s[i]; s[i] = s[j]; s[j] = t;
        i++; j--;
    }
}
```

11. C 语言中的内存对齐和字节序问题？

- 对齐：提升访问效率，避免硬件异常
- 字节序：大端/小端，网络通信需转换

12. 如何写一个安全的 strcpy 替代函数？

- 用 strncpy，手动加 '\0' 结尾，防止越界

13. C 语言中如何判断系统是大端还是小端？

- 通过联合体或指针访问最低字节判断

14. C 语言中的递归与迭代实现的区别？

- 递归：函数自身调用，易栈溢出
- 迭代：循环实现，效率高

15. C 语言中的静态变量和全局变量的区别？

- 静态变量：局部作用域，生命周期全局
- 全局变量：全局作用域，生命周期全局

三、知识点代码示例

malloc / calloc / realloc / free 的区别

malloc、calloc、realloc 和 free。malloc 按字节分配，初值不定；calloc 初始化为 0；realloc 调整已分配内存大小；free 释放内存，防止泄漏。

const 的各种组合

- `const int* p`：指向常量的指针，不能通过 p 修改值
- `int* const p`：常量指针，p 不能指向其他地址
- `const int* const p`：指向常量的常量指针

字节序问题 (endianness)

大端：高字节在低地址；小端：低字节在低地址。网络通信需注意。

安全 strcpy 示例

```
#include <stdio.h>
#include <string.h>

void safe_strcpy(char *dest, const char *src, size_t size) {
    strncpy(dest, src, size);
    dest[size - 1] = '\0';
}
```

如需详细解答某个问题，可随时补充说明